

Creator guide and reference

TextPhone for Voyage | Tested preview 0.6.0

Local testing, configuration and Voyage installation are separate workflows. Follow the complete merge and readback steps before a fresh game.

Contents	Page
A phone inside the story	2
Local development and testing	3
Merge your world export	4
Add the result in Voyage	5
Choose the settings that matter	6
Turn any app on or off	7
Add a custom app	8
Business lists and generated menus	9
Delivery rules belong to each app	10
Hidden apps and real unlock triggers	11
Connect the host systems	12
Build, edit and understand the runtime	13
API and persistence reference	14
Test the integration, not just the text	15
Optional improvements and sources	16
Verification: controls and configuration	17
Verification: scope and remaining limits	18
Vanilla compatibility and hidden Clock	19
Creator installation checklist	20
License and credits	21

A phone inside the story

TextPhone 0.6.0 | Creator and API guide

Give players a phone they can open in chat: installed apps, changing shortcuts, an App Store, nested business lists and narrator-generated service pages. A restaurant can lead to its menu, a job board to an offer, and a contact app to a conversation while the surrounding game continues.

Scripts own the installed-app list, shortcuts, selected app, configured directory entries, discovery filters, layout preference and reveal flags. The narrator presents those screens and generates page contents. The host game owns purchases, items, messages, appointments, contracts, quests and their consequences.

This is a single-player narrative interface. It is not a separate web app, a fixed-width phone image, a transaction engine or an authoritative alarm scheduler. The current adapter leaves phone state unchanged when it cannot identify exactly one player.

Player input	Result
phone	Open the installed-app screen
D	Open the app currently assigned D, from Home
phone store	Show available downloads
phone open APP NAME	Open an installed app
phone view PAGE	Browse a configured page or contextual detail
phone layout single / paired	Change the stored Home layout
close phone	Close the phone interface

Suggested first test: phone → phone store → phone open Maps → phone view nearby. Ordinary narration and the host clock continue; browsing can consume story time.

Local development and testing

Run the code on your computer; deploy separately in Voyage.

On this project's GitHub page, use Code > Download ZIP, extract it, and open a terminal in the extracted project folder containing package.json. Alternatively clone the repository with Git. The generic project is source and examples for adaptation, not a ready-to-use world package.

For local tests and builds, install Node.js with npm. package.json declares Node 20 or newer; the current release was checked locally with Node 24.19.0. No npm install, API key or Voyage login is needed for these offline checks. Node 20 compatibility is declared, not a claimed Node 20 test result.

Run the commands below from that project folder. npm test exercises scripted behavior with simulated Voyage checks, plus merge and installer event logic. It does not run Voyage, call a narrator model, or prove the final chat rendering. A successful run reports zero failed tests.

Edit config/generic-phone.json to adapt the apps and connections. Replace the synthetic Corner Cafe catalogue/location mapping with your own world entries before deployment. Then run npm test and npm run build again. Build refreshes both the partial mod and its matching HTML merger.

npm run build produces the files named on the next page. Testing/building does not install anything into a Voyage world. You can use the repository entirely for local development, then choose to deploy the result.

```
node --version
npm --version
npm test
npm run build
```

The HTML merger needs only a modern browser when already built. Node is for development and the command-line merge route; it is not the game runtime.

Merge your world export

One complete input world + one selected phone profile = one separate complete output.

First create a test copy: use More > Remix on a source world's detail page, or More > Duplicate on your own world. Confirm the copy's name/identity in Studio. Name it TEST__textphone__integration__YYYY-MM-DD__t01 using the actual date and a unique ID; record its purpose and cleanup condition.

In the copy's Studio workspace, choose Content at the top. Select the top-level Content entry in the left file tree (or Open Content in the breadcrumb) so the breadcrumb is just Content. If a form is shown, choose View JSON. The editor is labelled Studio initial GameState JSON editor. Select Download initial GameState JSON above the editor and preserve the complete export.

Create a local-worlds folder in the project and save the export as original.json inside it. This folder is ignored by Git; keep the backup private. A running game's save export or a single section such as Triggers is not this complete world definition.

Command-line route: after npm run build, run the command below. It reads build/generic-phone-mod.json, merges the phone triggers and narrator bridge, and writes local-worlds/with-phone.json. It refuses the original input as the output and refuses to overwrite an existing output; choose a fresh output filename on a later run.

Browser alternative: open build/generic-phone-installer.html from the extracted project in your browser. Under Original world JSON, choose original.json; select Prepare updated world, review the counts, then Download updated world JSON. Copy complete JSON is also available. The merger embeds the last built profile; rebuild after configuration changes.

Both routes use the same merger. It replaces only phone_mod_ triggers and prepends the supplied narrator bridge once in AI > Story > How to Use the Narrator. Other world fields are retained. A different existing phone bridge is a review error: compare it before attempting an upgrade.

```
node native/merge-world.mjs local-worlds/original.json  
  build/generic-phone-mod.json local-worlds/with-phone.json
```

Enter that command on ONE line in your terminal. The result is still a local file: nothing has been uploaded, installed or published yet.

Add the result in Voyage

Install the complete merged definition into the identified test copy.

1. Open with-phone.json in a text editor and copy its entire contents. For the browser route, use Copy complete JSON or open the downloaded file. Do not copy the partial build/*-phone-mod.json: it contains only phone trigger records.
2. Return to that test copy in Voyage Studio. Select Content, then the top-level Content entry so the breadcrumb is just Content. Choose View JSON if the editor is in form mode. Confirm you are in the complete initial GameState editor, not a single trigger or a running save editor.
3. Click inside the JSON text, press Ctrl+A (Cmd+A on macOS) while that editor is focused, and paste the complete prepared JSON. Use Save in the Studio toolbar. This is an editor replacement using the merged complete file, not a file upload or a Mods > Apply step.
4. Reload Studio, return to Content root and download the saved initial GameState JSON as local-worlds/saved-world.json. Run the comparison command below. Success means the saved complete data matches the prepared file; key order and indentation are ignored. A mismatch needs review before a gameplay test.
5. Start a fresh game from the test world. Name and record the save as a test where the interface allows it. Send phone, then phone store, then a shortcut actually displayed on Home. Open each configured app, test its directory and toggles, and continue ordinary play. Check stored state as well as the narrator's description of consequences. World-definition changes do not migrate existing saves. Native Mods > Apply is not verified for this release. The merger does not buy items, deduct currency or deliver messages; those outcomes remain with the host world and narrator. Remove unneeded test saves after preserving results; retain a test world only with a recorded reason.

```
node native/verify-world.mjs local-worlds/with-phone.json
  local-worlds/saved-world.json
```

Enter the comparison command on ONE line. Without Node, compare the prepared and saved complete JSON in a JSON-aware diff tool; counts alone cannot establish full equality.

06

Choose the settings that matter

Edit config/generic-phone.json, then rebuild. The starting apps are Messages, Maps, Food Delivery and App Store. Clock is configured but disabled; creators can enable it after connecting a suitable time system. Corner Cafe is an example: replace its catalogue entry with a business from your own world.

Setting	Meaning
columns	Default Home layout for a new phone: false=single, true=paired. Existing players keep their stored choice.
enabled	Per app: false omits it from the built mod; true or omitted enables it. Includes App Store.
initialInstalledIds	Enabled app IDs installed on a new phone. Include App Store when enabled.
storeAppId	The app that opens the scripted download catalogue.
appStoreVisibility	Per app: public, or triggered until revealed. This is separate from business discovery.
removable	Per app: false protects it from uninstall. App Store is always protected.
directory.storeListingMode	Per app: known, all_public, or curated.
delivery.blockedLocations	Per app: exact location/area exclusions and optional messages.
clockStorageKey	Optional name of a compact public clock record in host script storage.

Use stable IDs when renaming labels. The app label is what players type; the ID owns saved installation state. Reordering the catalogue does not intentionally reassign an existing valid shortcut. A removed configuration ID is pruned from that player's phone the next time it is processed.

Known means the world regards a named entity as known; it does not mean visited. The current native adapter rejects the visited listing mode. Device possession is not required and no invented device-availability flag is used.

Turn any app on or off

Every app, including App Store, supports `enabled:true` or `enabled:false`. Omitted means true for older configurations. Disabled apps are excluded during the build: their app instructions and business data are not shipped. A story reveal cannot enable an excluded app.

Use `enabled:false` to turn an app off. Use `enabled:true` with triggered visibility for an app that should exist but remain hidden until an event. Starting installation and player uninstall protection remain independent.

Disabling App Store removes its Home control and disables install/remove commands. Other preinstalled enabled apps still work. The Store must be in `initialInstalledIds` when enabled.

Rebuild both generated files, merge into a test world and start a fresh save. Existing world saves are not automatically updated. After an explicit script update, removed IDs are pruned at the next phone request; re-enabling does not reinstall them in that existing registry.

```
"enabled": false,
"appStoreVisibility": "public",
"removable": true
```

Intent	Settings
Off entirely	<code>enabled:false</code>
Downloadable	<code>enabled:true; visibility public; omit initial ID</code>
Start installed	<code>enabled:true; include initial ID</code>
Story unlock	<code>enabled:true; visibility triggered; omit initial ID</code>
Clock	Starts off. Enable after connecting a suitable host time system.

The switch controls this module. Independent world lore and narration may still mention the service. Do not treat it as a guaranteed narrative ban. The six new enablement tests run the generated scripts offline; the new master switch has not yet been exercised live.

Add a custom app

Add an object to the apps array with a unique id, label and order. This example adds a transit app that can be downloaded. To preinstall it in a new game, also add transit to initialInstalledIds.

```
{
  "id": "transit", "label": "Transit", "order": 20,
  "preferredShortcut": "T",
  "summary": "Local routes and ticket requests.",
  "pages": [
    { "name": "Routes", "kind": "generated",
      "guidance": "Show established nearby routes and fares.",
      "contextReader": "location" },
    { "name": "Request ticket", "kind": "generated",
      "startsActivity": true,
      "guidance": "Ask for destination; quote fare; await confirmation." }
  ]
}
```

Generated pages use phone view routes. An activity page uses phone choose request ticket. Opening the app itself does not select the activity.

The short summary appears in the Store and opening menu. Give it a concrete purpose: a name alone can lead the narrator to invent the wrong kind of service. Put general service rules in app.guidance and page-specific work in page.guidance. Keep the host's existing rules in force.

Use examples/transit-app.json for copying. To continue inside a generated menu, the player can use ordinary language. A generated phone view detail also retains the current app context, but it is model-mediated browsing rather than a new deterministic transaction command.

Business lists and generated menus

A directory is a curated set of candidate businesses attached to one app. All public means all entries marked public in that configured set; it does not scan every store in the world. The same world location can participate in several apps with different policies.

```
"directory": {
  "heading": "Stores", "contentLabel": "menu",
  "currency": "credits", "storeListingMode": "known",
  "entries": [
    {"id": "corner-cafe", "label": "Corner Cafe",
     "locationId": "Corner Cafe", "public": true,
     "description": "Tea, sandwiches and pastries."},
    {"id": "harbour-cafe", "label": "Harbour Cafe",
     "locationId": "Harbour Cafe", "public": true,
     "description": "Breakfast and hot drinks by the docks."}
  ]
}
```

Mode	Visible entries
known	Public entries whose exact locationId passes known-entity
all_public	Every public entry in this app's configured catalogue
curated	Public entries also listed in directory.curatedIds

For a curated list, add "curatedIds": ["corner-cafe"]. Set public:false to exclude a candidate from every listing mode. Add a page with kind:directory and optionally use openPage to open it immediately.

The listed number selects that business. The narrator then generates offerings and indicative prices. Numbers inside that generated menu are interpreted by narration and do not select another business. Back returns from menu to list, then from list to Home.

Descriptions are catalogue data. Explain what the business sells; do not hide engine instructions in them. Generated offerings and prices are not cached by this module. Use examples/store-app.json.

Delivery rules belong to each app

Configure exclusions independently for each app. An area-specific message takes precedence over a whole-location rule; otherwise the app message is used. Location and area IDs must match the host's native values exactly.

```
"delivery": {  
  "blockedMessage": "Food Delivery cannot reach this building.",  
  "destinationRequiredMessage": "Choose a delivery address.",  
  "blockedLocations": [  
    {"locationId": "Research Tower"},  
    {"locationId": "Research Tower", "areaId": "secure-lab",  
      "message": "Use the public reception pickup point."}  
  ]  
}
```

phone delivery check tests the player's current location and area in the currently open delivery app. It reports an exclusion, a missing destination, or that the destination is not excluded. The check does not place a new order. Not excluded is narrower than available: opening hours, stock, service coverage, funds, weather, fees and arrival estimates still belong to the host service. Natural-language ordering receives the app's exclusions as context. An alternative destination must be checked under those same rules.

This module can deterministically select and emit the refusal message. It cannot guarantee that every host model will obey it or veto every native purchase writer. Treat it as an integration policy, not an atomic financial guard.

There is no shared global blacklist. Giving Food Delivery a blocked location does not block a shopping app. Use `examples/store-app.json` for a complete editable app; configure both apps separately if both should be restricted.

Hidden apps and real unlock triggers

For an enabled app, set `appStoreVisibility:triggered` and leave it out of `initialInstalledIds`. The scripted catalogue omits it and the registry rejects a premature installation request. Live narration can still invent a successful installation: this is visibility/state control, not a guaranteed prose restriction. A reveal makes it available; it does not install it or grant account access.

```
const names = Object.keys(check({type: "player-level"}) || {});
if (names.length === 1) {
  const playerKey = JSON.stringify(names[0]);
  storage.phone_mod_unlocks = storage.phone_mod_unlocks || {};
  const grants = storage.phone_mod_unlocks;
  grants[playerKey] = grants[playerKey] || {};
  grants[playerKey].signal_vault = true;
}
```

Place that script in a creator-authored trigger with the actual quest, event or other supported condition that should reveal the app. The ID `signal_vault` must equal your app ID. Use the existing player key exactly; it includes JSON string quoting.

Alternatively, set `unlockKnownEntity` to an exact host entity name. A true known-entity result reveals the app when its phone is next processed. Reveal is persistent in this release; turning a grant back to false does not revoke a reveal that was already recorded.

Public download visibility, installed state and service authorization are separate. A public job app can still require membership before accepting work. Use app/page guidance to connect that service to the world's existing access rules.

`examples/hidden-app.json` and `examples/reveal-trigger.json` provide complete records. Replace the example quest condition with your world's real trigger. The adapter currently requires exactly one player; this example is not multiplayer attribution.

12

Connect the host systems

Connection	Current implementation
Player identity	Exactly one action string and one key from check(player-level); state is keyed by JSON.stringify(player name).
Location	check(party-location) and check(party-area); used by Maps and delivery checks.
Discovery	check(known-entity) for each configured exact locationId. No native historical-visit reader.
Clock	Disabled by default. Scripts cannot read the built-in clock. Optional clockStorageKey reads a creator-provided custom storage record for Time page context.
Orders and money	Host-generated transaction and state systems; the phone does not charge, pay or maintain a second ledger.
Messages and contracts	Host story/state systems; existing relationships, access rules and explicit choices remain relevant.
Living Continuity	May supply time and continuity through the host. The generic phone has no required LC dependency.

To enable Clock, find app id clock in your profile config and set enabled:true. Its supplied public visibility then allows downloading. Add clock to initialInstalledIds only if it should start installed in new games. Rebuild the mod and installer. LC installation alone does not configure the connection.

For an optional clock adapter, write a small public record such as storage.public_clock = {date:"2040-05-03", time:"09:20"}, then set clockStorageKey:"public_clock". Keep that record short: at most 1000 serialized characters are included. The host adapter must update it before the relevant phone turn; this module does not infer a civil date from game-tick.

Clock Alarms is a generated view, not a new scheduler. If your world requires dependable alarms, connect an existing authoritative event system and verify both the stored event and its later firing. An attractive generated alarm list is insufficient evidence.

There is no inventory-based device gate. Renaming a player can create a different phone key. Native multiplayer attribution is unsupported; offline per-name state isolation is not multiplayer support.

Build, edit and understand the runtime

```
node native/build-mod.mjs config/generic-phone.json build/custom-mod.json
node native/merge-world.mjs original.json build/custom-mod.json new-world.json
npm test
```

Source / public interface	Responsibility
native/build-mod.mjs : buildPhoneMod(config)	Validate configuration and build inert data records plus six native handlers. Returns {triggers}.
src/merge-phone-world.mjs : mergePhoneWorld(world, mod, bridge)	Return a separate complete world and preservation report; reject saves and conflicting bridges.
mod-context.mjs	Read configuration, attribute the player, restore/prune phone state, allocate stable shortcuts and build screen context.
mod-home / mod-marketplace	Home, layout, close/back, install, uninstall and hidden download visibility.
mod-services / mod-directory	Configured pages, activity choice, directory selection and generated-menu context.
mod-delivery / mod-follow-through	Per-app policy checks and context for natural or generated detail requests.

The builder emits `phone_mod_config_*` and `phone_mod_data_*` records as JSON inside inert script comments, read with `JSON.parse`. Runtime triggers use action-text conditions, planning phase and party scope. They use the documented storage, triggers, check, effects and log surfaces; they do not evaluate generated code. Only edit readable source/configuration, then rebuild. Deployment scripts remove indentation/comments to fit native limits; complete world exports stay pretty-printed. In Studio the generated records are under Content → Mechanics → Advanced → Triggers.

No npm dependencies are required. Node 20+ is for rebuilding and the CLI merger, not the Voyage runtime. The supplied HTML installer needs no Node installation, but its browser UI could not be live-tested in the authoring environment.

API and persistence reference

API or field	Contract
phone install / uninstall APP NAME	Case-insensitive exact app label; updates installation revision. Existing host orders/accounts are not deleted by script.
phone view PAGE	Browse a configured page. An unregistered detail continues inside the current app as model-mediated browsing.
phone choose PAGE	Invoke a configured startActivity:true page; ordinary page viewing does not invoke it.
phone shops / phone business NAME	Open the current app's eligible business list / select an exact listed label. phone restaurant NAME is an alias.
Back / phone back	Scripted directory menu→list→Home; from other app pages returns Home. Generated labels may claim a different destination.
phone_mod.players[playerKey]	Installed/revealed IDs, shortcuts, layout, app/page/directory, revision and lastHandled receipt.
phone_mod_unlocks[playerKey] [appld]	Creator grant:true reveals the download on the next processed phone action.

The receipt combines game tick and action text to prevent duplicate scripted navigation effects across handlers. A generated detail adds context without inventing a deterministic transaction. Natural conversation does not silently navigate the phone.

Configuration bounds: 32 apps, 12 pages per app, 128 businesses per app, unique lowercase ASCII app IDs up to 48 characters, unique short app labels up to 32 characters, and one-line summaries up to 160 characters. Page names use letters, spaces and hyphens, up to 40 characters. App/page guidance is bounded to 1800 characters each. Commands are limited to 400 characters. The builder also rejects any serialized trigger above 10000 characters.

Changing the starting app list or default layout affects new phone state. Existing state retains its installed IDs and chosen layout, apart from IDs removed from the current catalogue. Start a fresh game when changing profiles; legacy save migration is not provided.

Do not install two phone profiles together under the same namespace. Use the complete-profile merger to remove obsolete phone records. Keep unrelated trigger names outside phone_mod_.

Test the integration, not just the text

For every important change, compare four layers: the persisted phone state, the emitted instruction, the visible reply and the host’s resulting state. A correct registry does not prove the narrator showed the requested screen; a narrated purchase does not prove the inventory or currency is correct.

Use fresh saves for different starting defaults. Exercise both sides of each setting: paired/single, visible/hidden, before/after reveal, removable/protected, known/public/curated, blocked/allowed and area/default messages. Test a real player request after opening a page, and check that browsing did not accept, buy, hire or travel.

Retain failed trials. A later correct reply does not erase an omitted screen or invented record. Recheck when the model, world guidance, source version or Voyage renderer changes.

Known boundary	Practical response
Narrator omits or changes a screen	Try an exact app command and inspect state. Home can repeat a prior invented app, so returning Home is not a guaranteed recovery.
Generated controls outside the API	Use ordinary-language follow-up. phone view details has contextual routing; other invented commands are not guaranteed.
Lost labels or changing prices	Use clear word-bearing rows and explicitly confirm the quoted total. Inspect native consequences.
Invented alarms, accounts or profiles	Connect authoritative host records if required; generated prose alone is not a stored fact.
Multiplayer	Do not advertise support with this adapter.
Native mod application	Use the verified complete-export merge route until Mods → Apply is separately checked.

The final release verification sheet lists the actual test results and remaining limitations. No deterministic transaction veto, alarm scheduling guarantee, fixed-width handset frame or universal model compliance is claimed.

Optional improvements and sources

Creators can extend the phone without adding a new engine for every app. Start with a short purpose and one useful page, then add only the host connections the experience requires.

- Author a merchant catalogue per service, including location IDs, useful descriptions and discovery policy.
- Add a transit, journal, club, guild, weather, medical appointment or property app using generated pages and existing host systems.
- Reveal an app through a quest, invitation or known place; keep download visibility separate from account permission.
- Add an authoritative menu/cart/order or alarm adapter if exact persistence matters. This is extra implementation, not an undocumented built-in feature.
- Provide explicit profile candidates or service eligibility data where broad narrator guidance is too variable.
- Choose the starting apps and delivery messages that suit each world. A smaller catalogue usually makes the first phone interaction easier to read.

[Voyage Studio](#)

[Latitude Team Trigger Scripts Guide](#)

[Guide to Creator-Authored Quests](#)

[Voyage creation community](#)

Source basis: previously acquired Latitude documentation and production Studio/runtime observations, September 2026, engine 36. Prompt/context guidance was consulted before changing consumers. The module is independently authored integration code, not an official Latitude feature or guarantee. MIT covers the original TextPhone code and documentation. Third-party world material and assets remain separate and are not relicensed by this project.

Verification: controls and configuration

0.5.0 | Bounded production trials and offline tests

Tested on production Voyage engine 36 with a Living Continuity host; Story DeepSeek 3.2, Character Moderate, State Ultra. Results concern this integration and configuration. They are not a guarantee for another model or world.

Feature	Evidence and result
Single / paired Home	Saved setting and exact emitted rows pass. Paired rows rendered correctly; later single-column requests sometimes still rendered paired. Visible behavior is inconsistent.
Dynamic apps / shortcuts	Live install, remove and reinstall changed the registry. A new app whose preferred key was occupied received a different key; typing that key opened it.
Protected apps	Core Store refusal displayed correctly. Custom removable:false retained the app, but narration falsely claimed uninstall, including after a placement retry.
Known / public / curated	Controlled native lists exercised all three policies. Unknown businesses were filtered only in the known app; a separate public list and curated subset remained independent.
Per-app delivery	Live blocked/unblocked apps and distinct refusal messages passed. Area-specific wording took precedence. Whole-location fallback and missing-address branches also passed offline.
Hidden downloads	Live premature install refused; trigger grant made the download available; explicit install then succeeded. Immediate reveal narration was wrong. Known-entity reveal also passed offline.
Custom activity pages	Live shortcut and phone choose routed the intended page. It waited for a choice, but invented an invalid contact and controls. Routing passes; generated choice quality is limited.
Persistence	A fresh tab reopened the same saved game; layout, installed apps, reveals and shortcuts matched exactly.
Clock / location adapters	Native time/location pages were exercised. Optional clockStorageKey passed offline; no direct structured LC clock adapter is configured in the supplied profile.

Player layout commands are distinct from creator configuration options. Directory policy, protection, delivery and visibility are edited in configuration and rebuilt. Testing retains failed replies rather than counting correct script state as a complete visual pass.

Verification: scope and remaining limits

All nine world-profile apps and their configured pages were exercised. Trials covered messaging/jobs/calls, time/alarms, maps/directions, food menus/orders, shopping browse/cart/orders/stores, social profiles/messages, responder listings/hiring inquiries and restricted marketplace account/listings. Browsing and explicit follow-up were checked separately.

Two quote-to-confirmation purchase trials produced the expected native currency deductions only after explicit confirmation. No automatic job acceptance or responder hire was established in those browse/inquiry trials. Delivery arrival, long-term order memory, alarm firing, relationship outcomes and the complete employment lifecycle are host gameplay, not proven by these tests.

Observed model failures include omitted screens, retained paired rows after a single-column request, false removal notices, lost field labels, invented alarm defaults, unauthored account routes, invented controls and a wrong app inventory description that survived reinstall. A requested tracking quest was not created. No atomic transaction guard or authoritative generated ledger is provided.

The final source passes 72 offline checks: 23 current native-script checks, three complete-export merge checks, two synthetic installer-event checks and 44 retained prototype/component checks. Current repositories contain 19 generic checks and 38 customized checks, including two saved-readback verifier checks. These are offline checks, not native platform proof. The older 44 checks are historical evidence, not proof of the current native display.

The complete-export merge route was applied to the private test world and its complete saved JSON matched the generated result after reload. Existing host content was preserved. The same merge code and generic profile were tested offline. Native Mods → Apply and public mod publication were not validated.

The local HTML installer's merge logic and event flow passed synthetic checks. Browser policy blocked opening its local file in the authoring environment, so its live browser interface was not verified. The command-line route uses the same tested merger. A requested narrow viewport did not change the observed browser dimensions; mobile wrapping remains unverified.

Multiplayer attribution is unsupported. The adapter leaves phone state unchanged when there is not exactly one attributable player; independent-name offline checks do not establish multiplayer support. Older phone saves are not automatically migrated. Revalidate after changing the host instructions, model, routing, catalogue or platform renderer.

Use this as a configurable narrative interface with known presentation limits. Exact visible output and durable service records require further platform/host integration; those guarantees are not implied by the package.

Vanilla compatibility and hidden Clock

September 30, 2026: the distributed complete-export merger was applied to a fresh vanilla remix of source revision 176. The native saved full world matched the prepared output. All 63 host triggers, 227 locations and 19 starts were preserved; 23 phone records and the exact narrator prefix were added.

The new single-player vanilla save used Sonnet 4.5 Story with High character and state settings. This trial used the predecessor hidden-download configuration. The new master enabled switch is tested offline only; no native deployment or visual guarantee for it is claimed.

Initial Home displayed the four configured apps without Clock. An explicit phone install Clock request was refused in the saved registry and emitted Story instruction, but narration falsely claimed installation. The next Home repeated the invented Clock icon. Hiding an app controls the registry, not every sentence the model may generate.

The release has 38 world-project checks and 19 generic-project checks, including every app independently off, all apps off, App Store off, stale active selection, invalid flags and reveal resistance. Source configurations remain editable; generated artifacts include only enabled apps.

Check	Result
Installation/readback	Pass: exact host preservation and saved-output equality.
Hidden Clock state	Pass: absent from initial installed IDs and unrevealed download catalogue.
Hidden Clock visible refusal	Fail: narration claimed success despite refusal; Home carried it forward.
Creator enablement	Offline verified for both profiles. Live time accuracy/adapter not tested.
Other vanilla coverage	Meal directory and numbered menu selection worked; a permitted shopping-app installation opened its correct service menu. Final native state retained the installation, unchanged currency and one phone inventory item. Other service consequences were not retested here.

Native Mods > Apply, automatic save migration and multiplayer remain unverified or unsupported as stated. No new alarm scheduler or access to Voyage's native clock is provided.

Creator installation checklist

Check these steps on your own test remix. Checkmarks record your review, not a guarantee of model behavior.

- ☐ Preserve the original export; create and record a clearly named test remix.
- ☐ Choose one profile and replace synthetic catalogue entries where needed.
- ☐ Use exact native location/area IDs and distinct app names and IDs.
- ☐ Set starting apps, discovery policy and each app's delivery messages.
- ☐ Rebuild both the mod JSON and HTML installer after configuration changes.
- ☐ Paste the complete merged JSON at Content root; Save, reload and verify-world.
- ☐ Confirm original lore, NPCs, locations, quests and host triggers remain.
- ☐ Start a fresh save; open every app/page and test each chosen toggle.
- ☐ Test reveal before/after, install/remove, shortcuts and save/reload.
- ☐ Check narration AND saved state for consequential actions.
- ☐ Read the verification limits before sharing or using the profile.

License and credits

MIT for our work

MIT covers the original TextPhone code and documentation. Third-party world material and assets remain separate and are not relicensed by this project.

Not an official Latitude product. Voyage use remains subject to Latitude's applicable platform terms. The MIT license itself is unmodified.

MIT License

Copyright (c) 2026 Morberis

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

[Latitude Terms of Service](#)